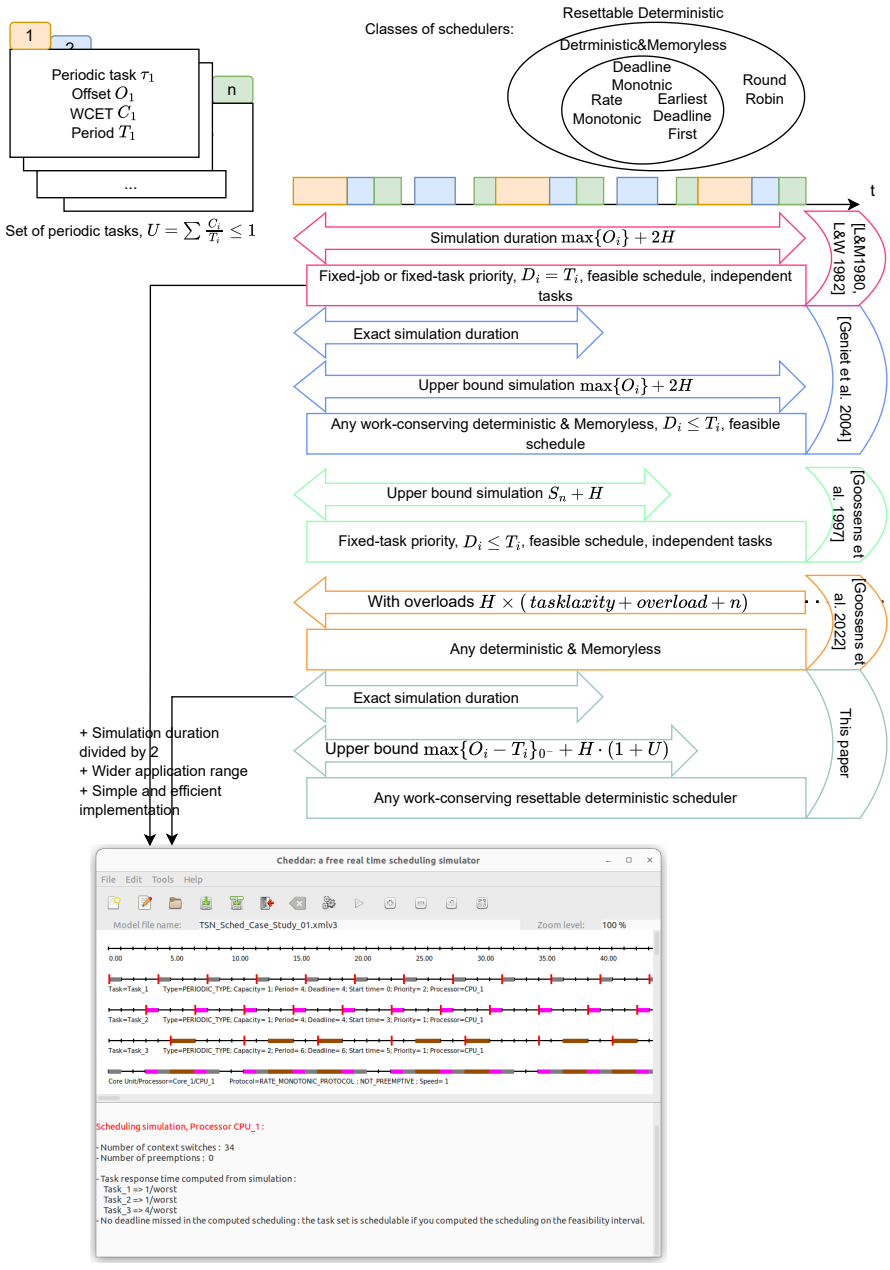
Graphical Abstract

Minimal simulation interval for periodic task schedulers

D. GUIDOLIN--PINA, E. GROLLEAU, F. SINGHOFF, H. N. TRAN



Highlights

Minimal simulation interval for periodic task schedulers

D. GUIDOLIN--PINA, E. GROLLEAU, F. SINGHOFF, H. N. TRAN

- Introduction of a new generic class of schedulers, called Resettable Deterministic Schedulers, which includes Deterministic & Memoryless (D&M) schedulers and includes non-D&M algorithms such as Round-Robin.
- Derivation of a strictly tighter upper bound than existing ones in the literature: $t_0 + H(1 + U)$, where $t_0 = \max_i\{O_i - T_i\}_{0^-}$, compared to the best known upper bound $\max_i\{O_i\} + 2H$.
- In the common case where all offsets are smaller than the periods, the upper bound simplifies to $H(1 + U)$.
- The result applies to all known work-conserving scheduling algorithms, supports task dependencies, arbitrary offsets and deadlines, and enables schedule computation during simulation, independently of feasibility. The proof is given for continuous time and is thus applicable to both continuous and discrete time-based simulations.
- The proposed algorithm to find the exact end of the first cyclic part is simpler and more efficient than the one found in the literature. It enables the Cheddar tool to be the first simulation framework providing the minimal required simulation duration.
- Experimental results show that, on average, the required simulation interval is divided by two compared to the classic bound.
- Our proposed upper bound is shown experimentally to improve the classic bound by a ratio of $(1 + U)/2$.
- The contributions have been implemented in the Cheddar tool.

Minimal simulation interval for periodic task schedulers

D. GUIDOLIN--PINA[a,b], E. GROLLEAU[b], F. SINGHOFF[c], H. N. TRAN[c]

[a] *LyRIDS, ECE Research Center, Paris, FRANCE*
[b] *LIAS, ISAE-ENSMA, University of Poitiers, Chasseneuil-du-Poitou, FRANCE*
[c] *Lab-STICC UMR CNRS 6285, University of Brest, Brest, FRANCE*

Abstract

Simulation-based analysis is widely used to study the temporal behavior of real-time systems. Its applicability relies on determining a finite simulation horizon beyond which the schedule becomes cyclic. Existing cyclicity bounds are often conservative or limited to specific scheduling algorithms, and many tools still rely on bounds that significantly overestimate the required simulation duration. In this paper, we introduce a generic class of schedulers, called Resettable Deterministic Schedulers. This class strictly includes Deterministic and Memoryless schedulers studied in the literature, while remaining compatible with non-memoryless algorithms such as Round-Robin. For this class, we derive an exact cyclicity result for work-conserving schedules of periodic task systems with processor utilization not exceeding one. We show that any such schedule becomes periodic after a finite transient phase, with period equal to the system hyperperiod (i.e., the least common multiple of task periods). We also provide an upper bound on the length of a cyclic schedule, which is strictly better than every known upper bound. The results apply to all known work-conserving scheduling algorithms, support arbitrary (*i.e.,* unrelated to periods) offsets and deadlines, and do not require feasibility assumptions. The bound is integrated into the Cheddar tool and significantly reduces the required simulation horizon in practice.

Keywords: Real-time systems, Cyclicity, Simulation-based analysis, Work-conserving schedulers

*Corresponding author
Email address: damien.guidolinpina@gmail.com (D. GUIDOLIN--PINA)

1. Introduction

Real-time systems are typically composed of recurrent tasks that release jobs either periodically or sporadically. These jobs are subject to stringent timing constraints. In domains where safety-critical real-time systems are predominant, such as automotive, aerospace, or power generation, industrial safety standards require a temporal validation of the system, commonly referred to as schedulability analysis.

Simulating a schedule consists in building a trace of the execution of a scheduler on a given task system, where tasks are characterized by given fixed parameters, generally execution times, periods, offsets, and deadlines. The main contribution of this paper is twofold. First, we derive an upper bound on the time horizon beyond which the sequence of scheduling events repeats cyclically. Second, we propose an algorithm that constructs an exact simulation and terminates after generating exactly one complete cycle of this cyclic behavior.

Scheduling simulation, although often not the most computationally efficient schedulability test, remains widely used because it can both assess schedulability and provide insight into the runtime behavior of the system. Unlike analytical schedulability tests, whose derivation is typically tied to a particular class of scheduling algorithms, the same simulation-based testing methodology can be applied to different scheduling algorithms, provided that the simulator implements the scheduling decisions and events associated with the algorithm under consideration.

However, simulation can be used as a sound schedulability test only under restrictive assumptions that ensure sustainability (see Baruah and Burns (2006); Nasri et al. (2018)). For instance, in uniprocessor systems, tasks must be independent and fully preemptive at all times for the worst-case scenario to occur in the simulation when tasks execute for their worst-case execution times (WCET).

Beyond schedulability testing, simulation is also commonly used to exhibit counterexamples demonstrating non-schedulability, or simply to illustrate representative scheduling behaviors. It can also be used for the optimal offline construction of feasible schedules, both in processor scheduling and in network scheduling, such as the construction of Gate Control Lists in Time-Sensitive Networks Steiner (2010), or time-driven scenarios.

Cyclicity results have been studied in the literature for more than forty years; however, most simulation tools investigated in Section 2.2 still rely on

the classic bound proposed in 1980 for Earliest Deadline First (EDF) scheduling in Leung and Merrill (1980), which was later shown to coincide with the bound for fixed-task-priority (FTP) schedulers Leung and Whitehead (1982). This bound limits the simulation horizon to $O_{\max} + 2H$, where $O_{\max}$ denotes the largest initial offset across all tasks, and H is the system hyperperiod, defined as the least common multiple (LCM) of the task periods.

Several results have generalized this upper bound to a broader class of schedulers, referred to as Deterministic and Memoryless (D&M). Informally, a scheduler is D&M if it takes the same decision whenever the system is in the same state. Here, the state of the task system is defined by the scheduling-relevant information at a given time, namely the remaining processing times of active jobs and the local clocks of tasks (see Definition 2.4 for a formal definition). Some classic schedulers, such as Round-Robin, maintain an internal state (they are said to be "hysteretic"), such as the state of the round-robin queue, and therefore do not conform to this definition.

In this paper, we consider a broader class of schedulers than the D&M schedulers studied in the literature: the Resettable Deterministic Schedulers (RDS). This class includes all D&M schedulers as well as schedulers that maintain an internal state, which is reset when the pending task queue is empty. For this class of schedulers, we show that, provided the processor utilization does not exceed 100%, any schedule produced by a work-conserving scheduler behaves cyclically with period H. We further show that the end of the first cycle corresponds to the first idle point occurring strictly after $t_0 + H$, where t_0 denotes the earliest instant after which all task releases repeat cyclically. Finally, we derive an explicit upper bound on the end of the cyclic part, given by $t_0 + H \cdot (1 + U)$, where U is the processor utilization.

We integrate this new exact simulation interval into the Cheddar scheduling simulator and propose an efficient implementation. Experimental results show that the proposed bound reduces the average simulation duration by a factor of two compared to the classic bound.

The paper is organized as follows. Section 2 introduces the necessary definitions and notations, and Section 2 reviews the related work. Section 3 positions our contribution and outlines the main ideas before Section 4 presents the new cyclicity bound. Section 5 gives an algorithm to compute the cyclic schedule and presents an experimental evaluation based on the Cheddar tool. Finally, Section 6 concludes the paper.

2. Definitions and notations

In this section, we introduce the system model as used in the literature and the background required throughout this paper.

We consider a set of periodic tasks denoted by $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$. Note that while the literature usually considers non-negative non-null integer parameters, in this paper, we consider the more general case of non-negative real values $\mathbb{R}_{\geq 0}$. Each task τ_i periodically issues jobs for execution and is characterized by:

- $O_i \in \mathbb{R}_{\geq 0}$: the *offset*, *i.e.,* the release time of the first job of the task (relative to a global time). We denote $O_{\max} = \max_{i \in \{1,\ldots,n\}} O_i$;
- $T_i \in \mathbb{R}_{\geq 0}$: the *period*, *i.e.,* jobs are released at times $O_i + kT_i$ with $k \in \mathbb{N}$, where $\mathbb{N}$ denotes the set of non-negative integers;
- $C_i \in \mathbb{R}_{\geq 0}$: the *Worst-Case Execution Time* (WCET), *i.e.,* the maximum amount of processing time required by any job of τ_i;
- $D_i \in \mathbb{R}_{\geq 0}$: the *relative deadline*, *i.e.,* the maximum time allowed for a job to complete after its release. If $\forall i, D_i = T_i$, the system has *implicit deadlines*; if $D_i \leq T_i$, it has *constrained deadlines*. When some tasks satisfy $D_i > T_i$, the system is said to have *arbitrary deadlines*, denoted by $D_i \# T_i$.

The *hyperperiod* of the system is $H = \mathrm{LCM}(T_1, \ldots, T_n)$. Also, we denote by $U = \sum_{i=1}^{n} \frac{C_i}{T_i}$ the processor utilization of the system.

Classic uniprocessor scheduling algorithms—such as Fixed-Job Priority schedulers (*e.g.,* EDF), Fixed-Task Priority schedulers (*e.g.,* Rate Monotonic or Deadline Monotonic), or dynamic-priority schedulers (*e.g.,* Least Laxity First)—are all *work-conserving*. A work-conserving scheduler never leaves the processor idle whenever at least one job is ready. The job selected by the scheduler executes until it either completes or is preempted.

In this paper, we use the term *simulation* to denote the construction of an execution trace, hereafter referred to as a *schedule*, produced by a scheduler on a given system with fixed task parameters (O_i, C_i, T_i). As mentioned in Section 1, such a trace can serve several purposes. It can be used to assess the feasibility of an online scheduler; in this context, the analysis typically requires sustainability with respect to variations of some parameters (e.g., execution times). A simulation can also be used to exhibit specific execution

scenarios, for instance, to illustrate particular behaviors or to demonstrate non-feasibility by constructing a counterexample. Finally, it can serve as a means to build an offline schedule. In this work, we focus on the analysis of the schedule itself, i.e., the execution trace generated under fixed parameters, independently of whether it is produced for the purpose of online or offline scheduling.

Although the schedule itself depends on the scheduling algorithm, any schedule produced by a work-conserving uniprocessor scheduler exhibits the same idle intervals, *i.e.,* intervals during which no job is pending, and the processor remains idle. This property does not hold for global multiprocessor scheduling, where both the amount and placement of idle times may depend on the scheduling algorithm, even when comparing global fixed job-priority (FJP) schedulers such as EDF and fixed task-priority (FTP) schedulers (see Goossens et al. (2016)).

As our model relies on task releases, we introduce the notion of local clocks associated with tasks. In this work, we adopt the local-clock model introduced in Garreau et al. (2023) and highlight its differences with previously proposed local-clock models.

Definition 2.1 (Local clock of a periodic task, from Garreau et al. (2023))**.** The local clock of task τ_i, denoted $\Theta_i : \mathbb{R}_{\geq 0} \Rightarrow \mathbb{R}_{\geq 0}$, transposes the value of the global clock t, increasing continuously over $[0, +\infty)$, in the local domain of the task. It is a continuous periodic function modulo T_i, taking its values in $[0, T_i)$, and releasing a job each time $\Theta_i(t) = 0$. It is defined on the open interval $(O_i - T_i, +\infty)$:

$$\Theta_i(t) = \begin{cases} (t - O_i) \bmod T_i & \text{if } t > O_i - T_i, \\ \text{undefined} & \text{if } t \leq O_i - T_i \end{cases}$$

Compared to previous local-clock models (*e.g.,* Goossens et al. (2016); Goossens and Masson (2022)), where each local clock Θ_i is undefined before O_i, the model of Garreau et al. (2023) defines local clocks from $O_i - T_i$ onward. This allows the collection of local clocks to exhibit periodic behavior earlier. Since each local clock Θ_i is periodic with period T_i, the composition $\Theta_1 \circ \cdots \circ \Theta_n$ is periodic with period H (see Knuth (2014), Section 4.5.2) and is defined for all $t > \max_i(O_i - T_i)$. Now, using these clocks, we can define the characteristic release functions.

Definition 2.2 (Characteristic release functions, adapted from Ladeira et al. (2022a))**.** The *release function* of task τ_i, denoted $r_i(t)$, models the workload generated by the periodic release of its jobs. It is a periodic function of period T_i defined as:

$$r_i(t) = \begin{cases} C_i, & \text{if } \Theta_i(t) = 0, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Next, we can define the remaining processing time of a processor.

Definition 2.3 (Total remaining processing time function, adapted from Ladeira et al. (2022a))**.** The *remaining processing time*, denoted $C_i : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$, is a right-continuous piecewise function representing, at each instant, the remaining processing time required to process active jobs of task τ_i. A discontinuity appears each time the release of a job occurs:

$$\forall t \geq 0, C_i(t) = C_i(t^-) + r_i(t) \tag{2}$$

where $C_i(t^-) = \lim_{\varepsilon \to 0, \varepsilon > 0} C_i(t - \varepsilon)$.
Without index, these quantities denote the total remaining processing time $C(t) = \sum_{i=1}^{n} C_i(t)$.

A job of τ_i is then *pending* at t only if its remaining processing time $C_i(t) > 0$. Only a pending task can be executed, and its quantity $C_i(t)$ *decreases linearly* while it is executing.

According to these previous definitions, we can now introduce the state of a scheduled task system and the scheduler class related to this latter notion.

Definition 2.4 (State of a scheduled task system, adapted from Goossens et al. (2016))**.** The state of a scheduled task system at time t, denoted $S(t)$, is defined by two vectors: the remaining processing times and the local clocks:

$$\begin{array}{rcl} S(t) & = & \{\mathcal{C}(t), \boldsymbol{\Theta}(t)\}, \\ \mathcal{C}(t) & = & \langle C_1(t), \ldots, C_n(t) \rangle, \\ \boldsymbol{\Theta}(t) & = & \langle \Theta_1(t), \ldots, \Theta_n(t) \rangle. \end{array} \tag{3}$$

Definition 2.5 (Deterministic and memoryless scheduler (D&M), from Goossens et al. (2016))**.** A scheduler is said to be *deterministic and memoryless* if and only if the scheduling decision at time t is unique and fully determined by the state of the system at time t.

2.1. Cyclicity in the uniprocessor case

We can classify cyclicity results into three categories: *(i)* specific bounds for a single scheduling algorithm (or a category thereof, such as FTP schedulers); *(ii)* generic bounds for work-conserving D&M schedulers; *(iii)* bounds based on the possible number of trajectories in the state space of a D&M scheduler, which are very pessimistic, as the state space increases when deadlines increase.

The classic cyclicity upper bound was introduced for independent tasks with implicit deadlines scheduled by EDF in Leung and Merrill (1980), and then demonstrated for FTP schedulers in Leung and Whitehead (1982). This classic cyclicity result shows that at the date $O_{\max} + 2H$ the state of the system is the same as at the date $O_{\max} + H$, hence the cyclic part.

$$\mathrm{UB}_{\mathrm{classic}}^{D_i \leq T_i, \mathrm{indep}, \mathrm{mono}, \mathrm{EDF}|\mathrm{FTP}} = O_{\max} + 2H. \tag{4}$$

It is important to note that the hyperperiod being the LCM of the periods, it is exponential, as $2^n \leq \mathrm{LCM}\{1, 2, \ldots, (n+1)\} \leq 4^{n+1}$ (see Chan and Norrish (2019)). The problem of schedulability of periodic tasks with offsets was shown to be co-NP-hard in Leung and Whitehead (1982), thus stating that simulation, while exponential, could still be used as an exact schedulability test.

A specific less pessimistic upper bound on the simulation interval, limited to feasible schedules generated by FTP schedulers, is given by $[0, S_n + H)$ in Goossens and Devillers (1997), where S_n is inductively defined on tasks ordered by priority level by:

$$\begin{aligned} S_1 &= O_1 \\ S_i &= \max\left(O_i,\, O_i + \left\lceil \frac{S_{i-1} - O_i}{T_i} \right\rceil\right) \\ \mathrm{UB}_{\mathrm{GD97}}^{D_i \leq T_i, \mathrm{indep}, \mathrm{mono}, \mathrm{FTP}} &= S_n + H \end{aligned} \tag{5}$$

The classic upper bound was also shown to be an upper bound for feasible schedules generated by FTP schedulers applied to tasks with arbitrary deadlines in Goossens and Devillers (1999).

$$\mathrm{UB}_{\mathrm{GD99}}^{D_i \# T_i, \mathrm{indep}, \mathrm{mono}, \mathrm{FTP}} = O_{\max} + 2H \tag{6}$$

An exact way to find the minimal cyclic part is given in Choquet-Geniet and Grolleau (2004) and can be applied to any work-conserving D&M scheduler, where tasks may be subject to mutual exclusion and precedence constraints. It shows that, during the transient phase—i.e., the finite initial

interval before the schedule becomes cyclic—any work-conserving scheduler may introduce idle intervals that occur within the cycle (i.e., before the higher confine of it) (referred to as acyclic idle times in the original paper). In a D&M feasible schedule, the cyclic part of length H starts after the last such idle interval (or at time 0 if none). In other words, the first window of size H during which the processor is idle for exactly $H(1-U)$ time units is the cyclic part, where U is the processor utilization factor. As pointed out in Garreau et al. (2023), a corollary is that both the transient phase and the following hyperperiod of a schedule contain only integral jobs, *i.e.,* a job started in the transient phase ends in this phase, and a job started in the first hyperperiod following the latest acyclic idle slot ends within this hyperperiod. The classic upper bound is tight, as it can be reached for some systems. The difficulty in applying this result comes from the fact that finding the latest acyclic idle time requires storing every idle time position until the first window of size H containing exactly $H(1-U)$ idle time units is found. It indeed requires storing every idle time position within the current hyperperiod, up to $O_{\max}+2H$ (computation of the id^{th} idle time in Algorithm 2). If we note δ the date of the end of the latest acyclic idle time, we have:

$$\begin{array}{lcl} \text{ExactCycleEnd}^{D_i \leq T_i,\text{mono,workcons,D\&M}} & = & \delta + H \\ \delta & \leq & O_{\max} + H \\ \text{UB}_{\text{GG04}}^{D_i \leq T_i,\text{mono,workcons,D\&M}} & \leq & O_{\max} + 2H \end{array} \tag{7}$$

Note the use of "less than or equal to" in the definition of the upper bound in Equation 7. The bound is not always attained: for some systems, the required simulation duration is only H, whereas for others it can be as large as $O_{\max}+2H$, depending on the existence and the position of the latest acyclic idle time.

The impact of accounting for preemption delays has been shown to significantly affect the cyclicity of schedules, as it may temporarily increase the processor demand and introduce additional idle intervals (see Goossens and Masson (2022)). Adding a task, adding one unit of preemption delay, and adding an extra time in the sum of the offset and the relative deadline compared to the period, all add an extra hyperperiod to the cyclic part. For the general case of any Deterministic and Memoryless scheduler, the cyclic part

is upper-bounded by:

$$\mathrm{UB}_{\mathrm{GM22}}^{D_i\#T_i,\mathrm{mono,D\&M}} = H \cdot (n+1) \cdot (\alpha_{\max}+1) \times \prod_{i=1}^{n} \big((O_i + D_i - T_i)_0 + 1\big) \tag{8}$$

where $X_0 = \max(X, 0)$, and $\alpha_{\max}$ is the preemption cost. This very pessimistic bound is trajectory-based, as it counts the possible trajectories of any feasible schedule in the state space, i.e., the possible sequences of system states over time induced by the schedule.

In Goossens (2008), the problem of cyclicity of feasible schedules for (m, k)-firm systems leads the author to consider all possible feasible schedules in which at least m out of every k jobs meet their deadlines, while the others are dropped, resulting in an upper bound that is a non-polynomial multiple of the hyperperiod.

To the best of our knowledge, all existing cyclicity results consider only feasible schedules. This paper is the first to address the case where jobs are allowed to miss their deadlines. In this setting, a job that misses its deadline is assumed to continue its execution until completion, regardless of the deadline miss.

To summarize, the classic bound, while initially proposed for EDF and FTP schedulers (Leung and Merrill (1980); Leung and Whitehead (1982)), was generalized to work-conserving deterministic and memoryless schedulers, and an exact bound for this generic class has been proposed. An upper bound specifically addresses FTP schedulers (see Goossens and Devillers (1997)). A result, considering all the possibilities in the state space, has been proposed as a very general upper bound. However, each additional unit of excess processing demand, or each additional deadline extending beyond the first hyperperiod, multiplies the simulation duration by H.

2.2. Cyclicity in simulation-based tools

Since the early 2000s, several scheduling analysis tools have been developed, including SimSo (Chéramy et al. (2014)), STORM (Urunuela et al. (2010)), Xoncrete (Brocal et al. (2010)), MAST (Harbour et al. (2001)), Cheddar (Singhoff et al. (2004)), TIMES (Amnell et al. (2003)), Yartiss (Chandarli et al. (2012)), and RealTss (Diaz et al. (2007)). These tools aim to support system designers by providing schedulability analysis and simulation features. Most are open-source, some are commercial, and several are no longer actively maintained.

TIMES relies on UPPAAL (David et al. (2018)) and models systems as timed automata. Schedulability is verified using model checking, while simulation mainly assists in debugging timed-automata models.

Other tools provide simulation features closer to classic real-time task models. MAST, for instance, targets distributed systems and, although primarily focused on response time analysis, also includes a scheduling simulator that produces execution traces and performance metrics.

Conversely, STORM, Yartiss, SimSo, and RealTss primarily focus on simulation-based analysis. Yartiss and SimSo address multiprocessor scheduling, the latter also considering shared hardware resources such as caches. STORM supports simulation of multiprocessor schedulers, with the simulation duration specified within the model. RealTss was designed to evaluate new scheduling policies. To the best of our knowledge, none of these tools automatically computes the simulation interval, which is instead defined by the user, either via a graphical interface (RealTss, SimSo, Yartiss) or directly in the model (STORM).

Only a few tools provide automated computation of the simulation interval. Xoncrete, initially developed for the XtratuM hypervisor, supports partitioned systems compliant with ARINC 653 (Prisaznuk (2008)). It generates offline schedules over the least common multiple of task sequence periods and seeks to minimize schedule length by adapting sequence periods. However, its applicability is restricted to partitioned systems compliant with or close to ARINC 653.

Cheddar computes the simulation interval prior to simulation using periodicity results from Goossens et al. (2016). Simulation can then serve as a schedulability test, provided the system model complies with the assumptions of the feasibility-interval analysis.

In summary, most existing scheduling simulation tools do not support automated cyclicity analysis or systematic computation of simulation interval bounds. Xoncrete provides automated computation but is limited to ARINC 653-like partitioned systems. Cheddar integrates several feasibility-interval methods, yet these may yield pessimistic bounds. This article proposes tighter bounds in the uniprocessor case.

3. Positioning of this work and main contributions

The exact bound proposed in Choquet-Geniet and Grolleau (2004) is strictly better than the classic upper bound. Nevertheless, most tools based

on simulation do not use the exact bound, and we believe that it is due to the technicality of finding the latest acyclic idle time, as well as its restrictive hypothesis (feasible schedule, constrained deadlines), and the fact that its upper bound is the same as the classic bound. We propose an efficient algorithm that does not require storing any idle time and can be used without storing any prior information about the schedule.

The bounds proposed for work-conserving D&M schedulers exclude some classic algorithms, such as round-robin schedulers, which rely on internal memory, namely, the current position in the ready queue used to determine the next task to be scheduled. This is an important limitation, since the default scheduling policy of the POSIX 1003.1 standard used in many real-time operating systems is SCHED_RR (see Grenier and Navet (2007)), which applies a round-robin policy to ensure a fair allocation of CPU time among tasks with equal priority. We generalize the exact bound to a broader class of schedulers, including round-robin and other scheduling algorithms.

We show experimentally that the average simulation duration obtained with the exact bound is half that of the classic bound. This is an important gain in time and memory for tools and is especially impactful in contexts where an offline schedule is implemented, for example, in the computation of Gate Control Lists used in IEEE 802.1Qbv Time-Sensitive Networks (see Garreau et al. (2023)).

Our new upper bound on the simulation duration is also the first one strictly better than the classic bound.

Finally, we show experimentally with the Cheddar tool that our new bound is twice as short on average than the classic upper bound given in Equation 4, while applied on a larger scope of task systems (arbitrary deadlines) and schedulers (RDS rather than D&M).

4. A new cyclicity bound

In this section, we formally define a new cyclicity bound and present the steps to obtain it. The section first introduces properties of local clocks and release functions related to periodicity. It then formally expresses the work-conserving condition and derives several related properties. Next, it presents the new bound in the general case before specializing it to a classic setting and discussing the resulting implications.

4.1. Periodicity of clock and releases

First, since we aim at characterizing cyclic behaviour, we explicitly highlight periodicity in the modelling of clocks and job releases.

Definition 4.1 (Onset of global periodicity)**.** Since offsets are not constrained a priori, we define t_0 as the time instant strictly preceding the onset of cyclic behaviour of the system, *i.e.,* the point from which the evolution of the local clocks becomes periodic.

$$t_0 = \max_{i=1,\dots,n} \left\{ O_i - T_i,\, 0^- \right\} \tag{9}$$

Note that, in the rest of the paper, t_0 always refers to this definition.

Lemma 4.1 (Periodicity of the local clocks)**.** The set of local clocks $\boldsymbol{\Theta} = \{\Theta_i\}_{i=1..n}$ is periodic with period H on $(t_0, +\infty)$, *i.e.,* $\forall i \in \{1, \dots, n\}, \forall t > t_0, \Theta_i(t + H) = \Theta_i(t)$. It is not periodic at or before t_0.

Proof. By Definition 2.1, if $t \leq O_i - T_i$, then $\Theta_i(t)$ is undefined. As a result, if $t \leq t_0$, there exists $k \in \{1, \dots, n\}$ such that $\Theta_k(t)$ is undefined and therefore not periodic. The set of clocks $\boldsymbol{\Theta}$ is then not periodic.

Now, if $t > t_0$, then $\forall i \in \{1, \dots, n\}, \Theta_i(t)$ is defined and periodic with period T_i. Therefore, according to (Knuth, 2014, Section 4.5.2), the set $\boldsymbol{\Theta}$ of the local clocks is periodic of period $H = \mathrm{LCM}(T_1, \dots, T_n)$. □

Please note that in the specific case where every offset is strictly less than its corresponding period, the set of clocks behaves cyclically from the initial instant 0; this is why, for this case, we take 0^-, as $(0^-, +\infty) = [0, +\infty)$.

Lemma 4.2 (Non-decreasing release functions)**.** The release functions (defined in Definition 2.2) are periodic and non-decreasing across successive hyperperiods. More precisely, for all $t \in \mathbb{R}$:

$$r(t) \leq r(t + H) \qquad \text{if } t \leq t_0, \tag{10}$$

$$r(t) = r(t + H) \qquad \text{if } t > t_0. \tag{11}$$

Proof. The release functions are triggered by the reset of the local clocks, and the total release is the sum of the individual tasks' releases. For $t \leq t_0$, it exists $k \in \{1, \dots, n\}$ such that $r_k(t) < r_k(t + H)$ since the clock is undefined. As a consequence, $r(t) < r(t + H)$. When $t > t_0$, all clocks are defined, and from Lemma 4.1 they are periodic with period H. Therefore, using Definition 2.2, we have the release functions have the same behaviour, *i.e.,* $r(t) = r(t + H)$. □

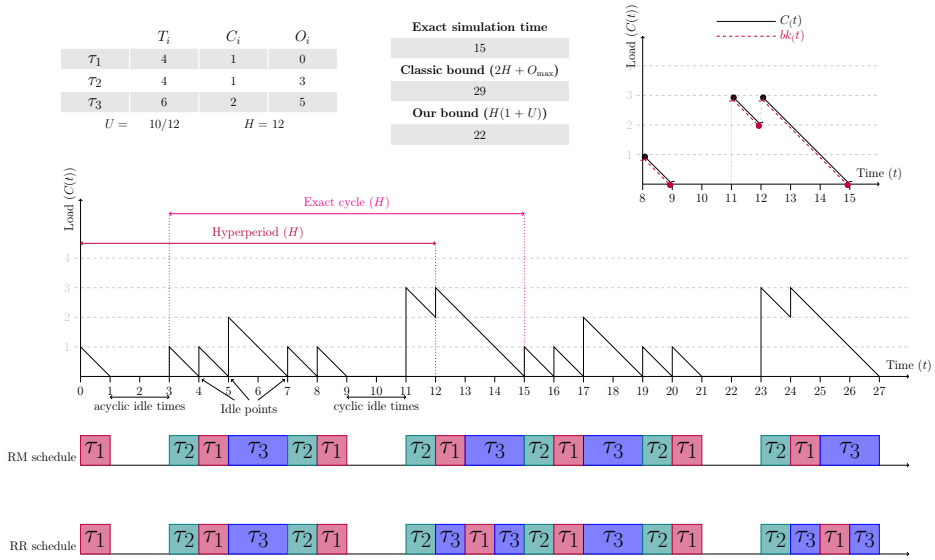


Figure 1: Illustration of the remaining processing function, the characteristic times and points, and two possible schedules (a Round-Robin and a Rate-Monotonic schedule) for a system composed of three tasks described in the table.

Note that according to this lemma, if $t \leq t_0$ then, $\sum_{s\in[t,t+H)} r(s) < H \cdot U$. If $t > t_0$, then $\sum_{s\in[t,t+H)} r(s) = H \cdot U$.

We now focus on the schedulers, and more specifically on the work-conserving strategy.

4.2. Properties of work-conserving scheduling algorithms

Before introducing the formal definitions and properties needed in the sequel, we illustrate the main notions with an example. We consider a simple system composed of three tasks, whose characteristics are provided in the table in the top-left corner of Figure 1.

Two schedules are shown: a Rate-Monotonic (RM) schedule and a Round-Robin (RR) schedule. Although the execution order of jobs differs between the two schedulers, both are work-conserving and therefore yield exactly the same evolution of the total remaining processing time, as illustrated by the plot (in the middle of Figure 1) showing the evolution of the remaining processing function $C(t)$.

Figure 1 also highlights the characteristic times and points associated with $C(t)$. Idle points correspond to instants at which the remaining processing time excluding new releases (which we call the *backlog*) is zero (for instance,

at $t = 4$, $t = 5$, or $t = 7$), whereas idle times denote intervals during which the remaining processing function is identically zero (for instance, for $t \in [1, 3)$ or $t \in [9, 11)$). Note that when the processor utilization reaches its maximum value ($U = 1$), the cyclic part of the schedule contains no idle time, and the maximum interval between two consecutive idle points is $H \cdot U$ (see Lemma 4.3). It is important to note that some idle times are acyclic idle times, *i.e.,* they occur only in the first hyperperiod. This is the case for the idle interval $[1, 3)$, since the corresponding interval in the next hyperperiod, $[13, 15)$, is not an idle time. These characteristic events are identical for both schedules, since they depend solely on the arrival pattern of jobs and on the work-conserving property of the scheduler, and not on the specific scheduling decisions.

Finally, to provide an overview of the proposed bound, the figure reports the simulation intervals obtained for this task system. The exact simulation interval is 15 time units, while the classic bound given by Equation 4 yields 29 time units. The upper bound derived in this paper, and proved in the subsequent sections, yields a bound of 22 time units. Finding the exact simulation interval using Choquet-Geniet and Grolleau (2004) requires storing and counting every idle time, while our method just finds the first idle point strictly after $t_0 + H = 0^- + 12 = 12^-$, *i.e.,* the first idle point found from time 12, which is at time 15.

The overview of the proof is as follows: first, we define the notion of busy periods and focus on idle points and idle times, since they are common to any work-conserving schedule. We then upper-bound the size of a busy period, which later allows us to derive our new upper bound on the simulation interval and to prove that there is at least one idle point per hyperperiod provided $U \leq 1$. Then, reasoning on idle times, we show that the backlog cannot decrease between any time t and $t + H$. This allows us to show that if the first idle point is found at $t + H$, for $t > t_0$, then an idle point is also found at t, establishing the cyclicity result.

Definition 4.2 (Backlog and Busy period)**.** The *backlog* of task τ_i, denoted bk_i, is the left-continuous piecewise function representing the amount of workload required to complete the active jobs of τ_i.

$$\forall t \in \mathbb{R}, bk_i(t) = C_i(t^-) \tag{12}$$

Without index, these quantities denote the total backlog $bk(t) = \sum_{i=1}^{n} bk_i(t)$. We say $[a, b)$ is a *busy period* if $bk(a) = 0 \wedge \forall t \in (a, b), bk(t) > 0 \wedge bk(b) = 0$.

It is worth noting that, according to Definition 2.3, $C_i(t) = bk_i(t) + r_i(t)$, *i.e.,* the backlog plus the release of a new job at t.

The top-right corner of Figure 1 shows a zoom on the interval $[8, 15]$ of the main plot. This zoom highlights the difference between the backlog and the remaining processing time. When no release occurs, for instance, for $t \in [8, 11)$, the backlog is equal to the remaining processing time. However, when releases occur, for instance at $t = 11$ or $t = 12$, the backlog corresponds to the value of the remaining processing time before accounting for the execution requirements of the newly released jobs. It therefore represents the amount of work before the arrival of new jobs, illustrating that $C(t) = bk(t) + r(t)$.

We can now formally introduce the notion of a work-conserving strategy.

Definition 4.3 (Work-conserving schedule)**.** A total remaining processing function is *work-conserving* if and only if it leaves the processor idle solely when the remaining processing time is zero, *i.e.,*

$$\begin{aligned} &\forall a \in \mathbb{R},\ \forall \delta > 0,\ \Big(\forall t \in [a, a+\delta),\ C(t) > 0\Big), \\ &\qquad bk(a+\delta) = bk(a) - \delta + \sum_{t \in [a, a+\delta)} r(t). \end{aligned} \tag{13}$$

From this definition, we can derive an upper bound on the maximum length of a busy period, which will be useful in subsequent proofs.

Lemma 4.3 (Maximum length of a busy period)**.** In the total remaining processing time function under a work-conserving strategy applied to a system with $U \leq 1$, the maximum length of a busy period is upper-bounded by $H \cdot U$.

Proof. The proof proceeds in two steps. First, we show that the busy period length is bounded by H. Then, using this result, we derive the tighter bound $U \cdot H$.

Let δ be the length of a busy period and let s be the start of this busy period.

1. Let us suppose that $\delta > H$. Then $bk(s) = 0$ and $bk(s + H) > 0$. Applying Equation 13, we have $H = \sum_{t \in [s, s+H)} r(t) - bk(s + H) > \sum_{t \in [s, s+H)} r(t)$.
   However, $\sum_{t \in [s, s+H)} r(t) \leq U \cdot H \leq H$. It shows by contradiction that $\delta \leq H$.

2. From Definition 4.2, $bk(s) = bk(s+\delta) = 0$, using Equation 13, we have $\delta = \sum_{t\in[s,s+\delta)} r(t) \leq \sum_{t\in[s,s+H)} r(t) \leq H \cdot U$.

□

Finally, the distinction between backlog and remaining processing time allows us to identify *idle points*, which play a central role in the remainder of this paper.

Definition 4.4 (Idle point and idle time, from Ladeira et al. (2022a))**.** An *idle point* occurs at time t if $bk(t) = 0$. An *idle time* starts at time t if $C(t) = 0$ and ends at the smallest $t' > t$ such that $r(t') > 0$.

Idle points and idle times are key to the cyclic behavior of remaining processing time functions under a work-conserving strategy. Therefore, we need to characterize exactly what the idle time is, more especially since we consider that our model parameters are defined in the real-number domain.

Lemma 4.4 (Total idle time over an interval)**.** In the total remaining processing time function under a work-conserving strategy, the total idle time on any interval of duration δ from any time a, denoted $\mathcal{I}(a, a+\delta)$, is given by the length of the interval minus the total request in the interval plus the difference of backlog between the start and end of the interval, more formally

$$\mathcal{I}(a, a+\delta) = \delta + bk(a+\delta) - bk(a) - \sum_{a\leq t<a+\delta} r(t). \tag{14}$$

Proof. If the interval $[a, a+\delta)$ is a busy period, then Equation 13 applies and $\mathcal{I}(a, a+\delta) = 0$, which directly satisfies 14. Otherwise, the interval $[a, a+\delta)$ can be decomposed into a finite sequence of busy and idle sub-intervals. On each busy sub-interval, Equation 13 applies, and the length of the sub-interval equals the total amount of released work over it. On each idle sub-interval, no release occurs. Accounting for all sub-intervals, and applying Equation 13 to the first and last busy sub-intervals when they are only partially contained in $[a, a+\delta)$, the total length δ of the interval can be expressed as the sum of the idle time $\mathcal{I}(a, a+\delta)$ and the net variation of the backlog minus the total released work. Rearranging terms yields

$$\mathcal{I}(a, a+\delta) = \delta + bk(a+\delta) - bk(a) - \sum_{a\leq t<a+\delta} r(t),$$

which concludes the proof. □

Finally, we can also prove a useful property on the backlog, which is that it cannot decrease in subsequent hyperperiods.

Lemma 4.5 (Non-decreasing backlog on subsequent hyperperiods)**.**

$$\forall t \in \mathbb{R}, bk(t+H) \geq bk(t). \tag{15}$$

Proof. It is trivial if $bk(t) = 0$, since $\forall t \in \mathbb{R}, bk(t) \geq 0$.

For the case $bk(t) > 0$, then t is in a busy period, so let t_c be the starting instant of this busy period, *i.e.,* $bk(t_c) = 0$ and $\forall s \in [t_c, t), C(s) > 0$. Since $[t_c, t)$ is a busy period, we can apply Equation 13 $bk(t) = \sum_{t \in [t_c,t)} r(t) + bk(t_c) - (t - t_c)$. As $bk(t_c) = 0$, then $bk(t) = \sum_{s \in [t_c,t)} r(s) - (t - t_c)$.

Now, let us consider the interval $[t_c + H, t + H)$. From Equation 14, we have

$$bk(t+H) = bk(t_c+H) - (t - t_c) + \mathcal{I}(t_c+H, t+H) + \sum_{s \in [t_c+H,\, t+H)} r(s) \tag{16}$$

$$bk(t+H) = bk(t) + bk(t_c+H) + \mathcal{I}(t_c+H, t+H) + \sum_{s \in [t_c+H,\, t+H)} r(s) - \sum_{s \in [t_c,\, t)} r(s) \tag{17}$$

But, by definition of the functions, $\mathcal{I}(t_c + H, t + H) \geq 0, bk(t_c + H) \geq 0$, and from Equation 10 and 11, we have that $\sum_{s \in [t_c+H,\, t+H)} r(s) \geq \sum_{s \in [t_c,t)} r(s)$

As a consequence, $bk(t + H) \geq bk(t)$. □

All these definitions and properties allow us to introduce a new cyclicity bound based solely on the total remaining processing time under a work-conserving strategy.

4.3. Properties of a work-conserving schedule produced by a RDS

Before giving the new cyclicity bound, we formally define a class of schedulers: the *resettable deterministic schedulers* (RDS) to which our cyclicity results apply.

Definition 4.5 (Resettable deterministic scheduler - RDS)**.** A resettable deterministic scheduler (RDS) may have an internal state, but this state is reset to a value when the total backlog is zero, *i.e.,* at an idle point. Being deterministic, it makes the same scheduling decisions when presented with the same sequence of job releases since the previous idle point.

Formally, the state at time t of a system scheduled by an RDS is defined by $\mathcal{S}(t) = \{\mathcal{C}(t), \mathbf{\Theta}(t), I_\sigma(t)\}$, where $\mathcal{C}$ is the vector of remaining processing times of the tasks, $\mathbf{\Theta}$ is the vector of local clocks, such as defined in Definition 2.4, and $I_\sigma(t)$ is the internal state as obtained following the schedule σ produced since the last reset of the internal state of the scheduler. The resetting constraint requires that there exists a reset value I_0 such that $\forall t,\ \sum_{i=1}^{n} bk_i(t) = 0 \ \Rightarrow\ I_\sigma(t) = I_0$. Given the same history of job arrivals and starting from the same state, an RDS produces the same schedule and reaches the same internal state.

From this more general definition compared to D&M, we can only show the equivalence of a system scheduled by an RDS starting from idle points.

Let us now introduce the cyclicity result.

Lemma 4.6 (Cyclicity of work-conserving RDS schedules)**.** A work-conserving RDS schedule of a system with $U \leq 1$ behaves cyclically over the interval $[t_c, t_c + H)$, with $t_c + H$ the first idle point strictly after $t_0 + H$. Moreover, $t_c \leq t_0 + H \cdot U$.

Proof. First, since a busy period cannot exceed $H \cdot U$, the idle point found at $t_c + H$ necessarily occurs at most $H \cdot U$ after $t_0 + H$, therefore, $t_c \leq t_0 + H \cdot U$. Moreover, since $t_c + H$ is an idle point, so is t_c, as by Lemma 4.5, $bk(t_c + H) \geq bk(t_c)$ and bk is a non-negative function. As a result, since $t_c \geq t_0$, by Lemma 4.1, the same requests occur in $[t_c, t_c + H)$ as in $[t_c + kH, t_c + (k+1)H)$ where k is non-negative integer. Therefore, we observe the same state of the system and the scheduler is in its reset state both at the date t_c and $t_c + H$, which, by definition of a work-conserving RDS, will generate the same schedule on $[t_c, t_c + H)$ as in the subsequent hyperperiods. □

For more convenience, we reformulate Lemma 4.6 as a theorem independent of the paper's specific notations. Lemma 4.6 first identifies $t_c + H$, the end of the first cyclic hyperperiod, as the first idle point strictly after $t_0 + H$. Since the cyclic behavior has period H, the beginning of the cyclic phase is obtained by subtracting one hyperperiod, yielding $t_c = (t_c + H) - H$. The following theorem allows us to find an upper bound on the end of the cyclic phase.

Theorem 1 (General cyclicity)**.** *A work-conserving RDS schedule of a system with $U \leq 1$ becomes cyclic with a period H. The end of the cyclic phase*

is the first idle point strictly after $t_0 + H$, where $t_0 = \max_i \{O_i - T_i\}_{0^-}$. An upper bound of the end of this cyclic phase is given by

$$t_0 + H \cdot (1 + U). \tag{18}$$

Proof. Reformulation of Lemma 4.6 □

This theorem is the general case. It provides an upper bound without any additional restriction on the task offsets, and therefore applies to arbitrary task systems. We also derive a simplified upper bound for a classic, widely studied case in which task offsets are smaller than their periods. This restricted setting is of particular interest in several practical and theoretical contexts: for task systems, it includes well-known offset-free models such as those studied in Goossens (2003); Ladeira et al. (2022b); for communication systems, similar assumptions arise in TSN scheduling approaches such as Time-Aware Shaper (TAS) (Steiner (2010), Garreau et al. (2023)).

Corollary 4.1 (Cyclicity of systems with constrained offsets)**.** Consider a task system such that $\forall i, O_i < T_i$. Then any work-conserving RDS schedule of a system with $U \leq 1$ is cyclic with period H. The end of the cyclic part is the first idle instant at or after H. An upper bound on the end of this cyclic part is $H \cdot (1 + U)$.

Proof. It is Theorem 1 with $t_0 = 0^-$. □

Having established the cyclic behavior of work-conserving RDS schedules, we now relate this result to the feasibility analysis of real-time systems. Up to this point, the paper has not required a distinction between online and offline scheduling: a simulation is a trace of a scenario with fixed parameters. We therefore define feasibility in the offline sense to make explicit that, when we refer to the feasibility of a simulation, it concerns feasibility under fixed parameters, that is, assuming no variation in any parameter.

Definition 4.6 (Feasibility)**.** An offline schedule is feasible if, considering the tasks' parameters (WCET, period, offset, deadline, etc.), every job released in $[0, +\infty)$ completes execution at or before its deadline.

Using the cyclicity result, we can now characterize feasibility by restricting the analysis to a finite time interval.

Lemma 4.7 (Feasibility of a schedule)**.** If no deadline is missed before the end of the cyclic part in a work-conserving RDS schedule, with $U \leq 1$, then no deadline will ever be missed in the schedule.

Proof. As the cyclic part starts and ends at an idle point, every job started within the cyclic part of the schedule is finished by the end of the cyclic part. It can therefore be compared to its individual deadline by the time the end of the cyclic part is reached. □

While the cyclicity result provides a finite horizon for schedulability analysis, it also raises questions regarding the interpretation of the idle times occurring before the cyclic part is reached.

4.4. Discussion about the acyclic idle times

When we came across this new result, our first question concerned the acyclic idle times: why is it no longer necessary to determine their exact locations? For simplicity, consider a system for which $t_0 = 0^-$. At H, if we do not observe an idle point, the backlog $bk(H)$ is the length of acyclic idle times that occur in the interval $[0, H)$ (from Lemma 4.4 and given that there are $H(1-U)$ cyclic idle times per hyperperiod). This "extra" backlog will occupy the first $bk(H)$ idle times that were not occupied during the first hyper period.

In Figure 1, we have $bk(H) = bk(12) = 2$. $[1, 3)$ is idle, but $[H+1, H+3) = [13, 15)$ is busy because of the backlog at the hyperperiod. Mechanically, the backlog corresponding to remaining workload and pending job releases at the end of a hyperperiod carries over to the next hyperperiod and may delay the occurrence of idle points until this workload has been fully processed. This interaction explains the emergence of cyclic behavior in the schedule.

Why was this issue not identified earlier? It stems from the definition of the local clocks. Under the previous definition, even when offsets were constrained, tasks were only proven to exhibit periodic behavior after time $O_{\max}$. With the new local clock definition (Definition 2.1), each local clock is periodic from time $O_i - T_i$. In Figure 1, the local clock of τ_3 was previously considered periodic starting from time $O_3 = 5$, which prevented the cyclic part of the schedule from beginning earlier. With the revised definition, however, the local clock is periodic from $O_3 - T_3 = -1$, *i.e.,* from the very beginning of the schedule.

Applying Theorem 1, or directly Corollary 4.1, the upper bound is thus $H \times (1 + U) = 22$, which is strictly lower than the previously known upper bound $O_{\max} + 2H = 29$.

5. Computing the schedule in practice

In this section, we first propose a new algorithm to compute the smallest schedule before testing it on several experiments.

5.1. Algorithm to compute the smallest schedule

To compute the cyclic schedule in practice, we present a very simple algorithm based on Theorem 1 to determine the end of the cyclic phase. The algorithm is presented in Algorithm 1.

Algorithm 1 Computation of the cyclic schedule

Require: $U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$
Require: A work-conserving RDS
Ensure: Schedule up to the cyclicity point
$t_0 = \max\left(\max_{i=1,\dots,n} \{O_i - T_i\}, 0^-\right)$
$H = LCM(T_i)$
Compute the schedule up to the first idle point strictly after $H + t_0$
▷ The latest window of size H of the schedule is the cyclic part

If the scheduler works in discrete time, the definition of t_0 has to be adapted as follows: it is the maximum value between the immediate predecessor of 0 and $\max_{i=1,\dots,n} \{O_i - T_i\}$. In the common case of unit-time granularity, this gives $t_0 = \max\left(\max_{i=1,\dots,n} \{O_i - T_i\}, -1\right)$.

Except for addressing a broader context and a larger set of schedulers, our algorithm (Algorithm 1) computes the same cyclic part as Algorithm 2 proposed in Choquet-Geniet and Grolleau (2004). Notably, however, Algorithm 2 requires memorizing the positions of all the idle times encountered within the previous hyperperiod and iterating until a hyperperiod with exactly $H \cdot (1 - U)$ idle times is found. Our algorithm is simpler and more efficient, as it does not require storing any information about previous events while building the schedule. Moreover, our algorithm has fewer requirements since it does not require feasibility, supports arbitrary deadlines, and addresses the very wide class of RDS.

Algorithm 2 Computation of the cyclic schedule as proposed in Choquet-Geniet and Grolleau (2004)

Require: $\forall i, D_i \leq T_i$
Require: $U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$
Require: A work-conserving D&M scheduler
Require: The schedule is feasible
Ensure: Exact cyclic schedule
  $H = LCM(T_i)$
  $t1 = 0$
  repeat
    Continue the schedule on $[t1, t1 + H)$
    NbIdle $= \mathcal{I}([t1, t1 + H))$
    if NbIdle $> H \cdot (1 - U)$ **then**
      $id =$ NbIdle $- H \cdot (1 - U)$
      $t1 =$date following the id^{th} idle time in the schedule $[t1, t1 + H)$ ▷ Requires computing the number and positions of the idle times in the last hyperperiod
    end if
  until NbIdle $= H \cdot (1 - U)$ or missed deadline
    ▷ The latest window of size H of the schedule is the cyclic part

5.2. Experiments

This section aims to highlight the benefits of our main contributions, namely the proposed bound and the exact computation of the simulation interval, both introduced in Theorem 1. Using Cheddar and Algorithm 1, the exact simulation interval is compared with the classic bound (Equation 4), our proposed bound (Equation 18), and the lower bound ($t_0 + H$). Note that $t_0 + H$ is a lower bound, as follows from Lemma 4.1, a schedule cannot see a cyclic part at or before this date.

As mentioned, the experiments reported in this paper rely on the Cheddar scheduling analysis tool, an open-source software distributed under the GNU GPL V2 license. The simulation capability required for this study will be integrated into an upcoming public release of Cheddar (Cheddar 3.4), planned for Summer 2026.

In the meantime, to ensure full reproducibility, we provide an artifact consisting of a standalone preview binary of Cheddar together with all task-set configurations used in the experiments and the corresponding obtained results. The artifact is available at: `https://beru.univ-brest.fr/svn/CHEDDAR/artefacts/JSA26/`.

A total of 15,000 task-set configurations are generated with the following parameters:

1. The number of tasks is uniformly set to 8, 16, or 24;
2. The utilization U ranges from 0.4 to 1;
3. Task periods are randomly generated following the methodology described in (Ladeira et al., 2022a, Section 6.1) to control the value of H and maintain it lower than 1,209,600, while avoiding harmonic periods;
4. Execution times are obtained using the approach proposed in Griffin et al. (2020) which is generating unbiased distributions of tasks utilization factors;
5. Task offsets are randomly selected either in the interval $[0, T_i)$ or in $[0, 10 \cdot T_i)$;
6. While any conservative RDS scheduler generates the same cyclic part, we tested several scheduling policies for the purpose of testing our implementation. We used non-preemptive/preemptive Rate Monotonic and non-preemptive/preemptive EDF, and SCHED_RR, which gave, as expected, the same results. Note that jobs missing their deadline are not discarded.

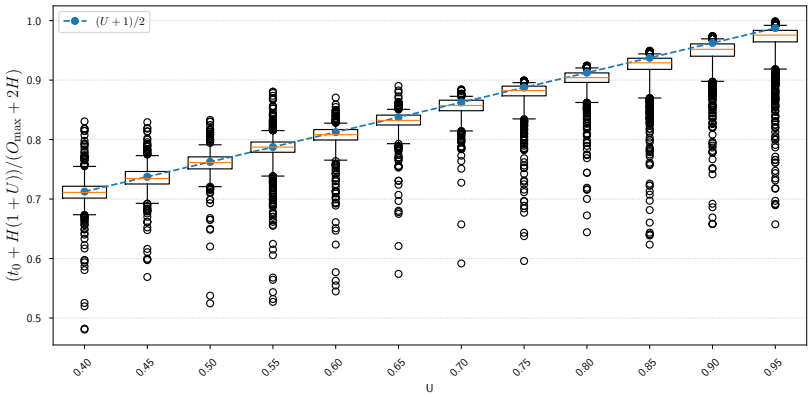


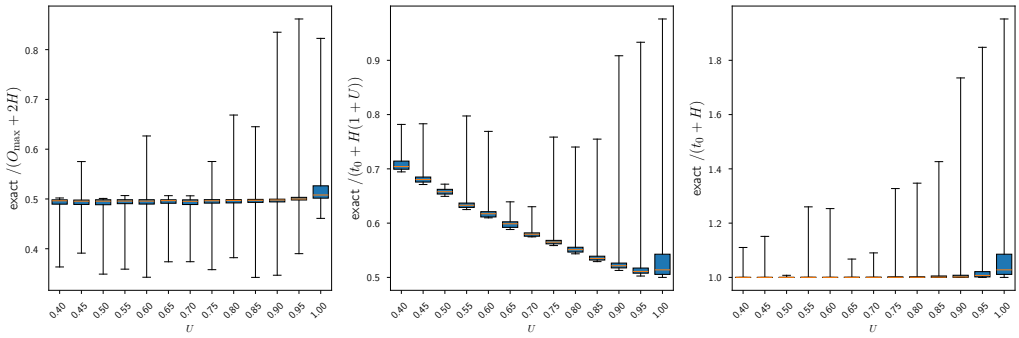
Figure 2: Distribution of the ratio between the proposed (Equation 18) and the classic bound (Equation 4) relative to system utilization.

We then draw distribution graphs of different ratios according to the utilization (U). Each box-plot contains, on average, 567 values (from 125 to 1347 because generating limit cases is more complicated than the others). Note that, in the generated graphs, the number of tasks does not significantly influence the results; therefore, the results for 8, 16, and 24 tasks are presented together in the same graphs. In contrast, the offset configuration has a noticeable impact, which motivates the presentation of different plots. As illustrated in the following figures, the configuration with offsets bounded by $O_i < 10 \cdot T_i$ exhibits a larger dispersion of the results, due to the individual impact of the maximum offset $O_{\max}$.

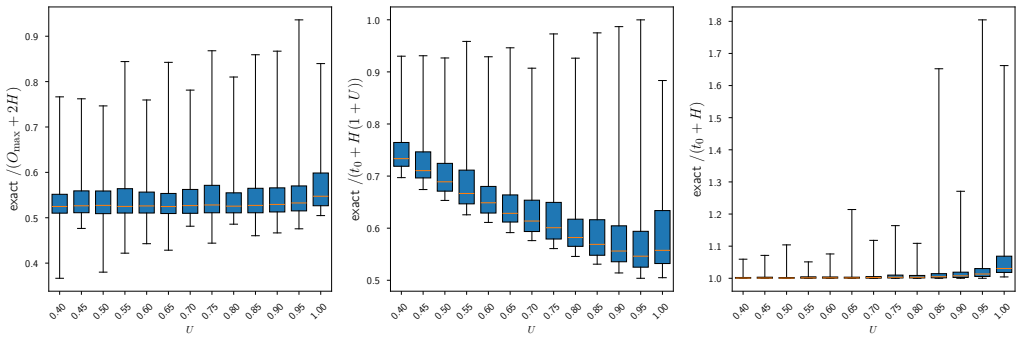
Figure 2 presents the distribution of the ratio between our upper bound and the classic bound, according to the system utilization. As expected, this ratio is centered around the line $(1 + U)/2$, illustrating that our bound is strictly tighter than the classic bound.

Next, each configuration is simulated using Cheddar and Algorithm 1. Figure 3 presents the distribution of the obtained results. For each sub-figure, the left plot shows the ratio between the exact simulation interval and the classic bound, the middle plot shows the ratio with the proposed bound, and the right plot shows the ratio with the lower bound, all as a function of the utilization U.

Regarding the comparison between bounds and exact values, the classic bound overestimates the cycle time by approximately a factor of two (see the left-hand-side sub-figures of Figure 3), whereas our upper bound depends on



(a) $O_i < T_i$



(b) $O_i < 10\ T_i$

Figure 3: Distribution of ratios of the exact simulation interval compared to, from left to right, the classic bound (Equation 4), our proposed bound (Equation 18), and the lower bound $(t_0 + H)$.

the processor utilization U (see the middle sub-figures of Figure 3). More precisely, our upper bound increases with U and converges to the classic bound as U approaches 1. Finally, in the vast majority of cases, the cycle time is close to the lower bound $t_0 + H$, as illustrated in the right-hand-side sub-figures of Figure 3. However, rare cases can be observed in which the cycle time approaches the upper bound; these occur more frequently at higher utilization levels.

6. Conclusion

In this paper, we propose an exact cyclicity result that is more general with respect to the underlying assumptions than existing approaches. In

particular, it relaxes several hypotheses on the task system (feasibility is not required, deadlines may be arbitrary, and offsets may be arbitrary) and on the scheduler, as the class of Resettable Deterministic Schedulers (RDS) strictly includes the Deterministic & Memoryless (D&M) schedulers considered in the state of the art. Moreover, the proposed bound is simpler to compute and to implement than the previous exact bound introduced in Choquet-Geniet and Grolleau (2004). Our upper bound is also strictly tighter than the classic bound $O_{\max} + 2 \cdot H$, which is still commonly used as an upper bound in exact cyclicity analyses.

We define $t_0 = \max_i \{O_i - T_i\}_{0^-}$, where O_i and T_i denote the offset and period of task τ_i, respectively, as the earliest time after which the set of task releases becomes periodic. Note that when all task offsets are strictly smaller than their periods, we have $t_0 = 0^-$. For a set of periodic tasks with utilization $U = \sum_{i=1\ldots n} \frac{C_i}{T_i} \leq 1$, where C_i denotes the worst-case execution time of task τ_i, and scheduled by a work-conserving RDS, any schedule eventually behaves cyclically. More precisely, the end of the first cyclic part occurs at the first idle point (i.e., a time instant with zero backlog) strictly after $t_0 + H$, where $H = LCM_{i=1\ldots n}(T_i)$. This instant is guaranteed to occur no later than $t_0 + H \cdot (1 + U)$.

Experimentally, we show that, on average, the simulation duration is close to only half the time given by the classic upper bound, which is still widely used by users of simulation-based tools. This is an important gain, as the hyperperiod can grow exponentially with the periods of the tasks. Moreover, our upper bound, which is the first upper bound smaller than the classic upper bound on the simulation duration, is approximately a factor of $\frac{U+1}{2}$ smaller than the classic upper bound.

As a perspective, we emphasize the main limitation of this result: the task system is assumed to be static. In particular, the proposed analysis does not directly extend to mixed-criticality task systems, nor to systems in which jobs may be dropped upon deadline misses. Furthermore, the assumption $U \leq 1$ excludes overload situations; for instance, cache-related preemption delays and other sources of execution-time inflation are out of the scope of this work and are left for future research.

References

Amnell, T., Fersman, E., Mokrushin, L., Pettersson, P., Yi, W., 2003. Times: a tool for schedulability analysis and code generation of real-time systems,

in: International Conference on Formal Modeling and Analysis of Timed Systems, Springer. pp. 60–72.

Baruah, S.K., Burns, A., 2006. Sustainable scheduling analysis, in: Proceedings of the 27th IEEE International Real-Time Systems Symposium (RTSS 2006), IEEE. pp. 159–168. doi:10.1109/RTSS.2006.47.

Brocal, V., Masmano, M., Ripoll, I., Crespo, A., Balbastre, P., Metge, J.J., 2010. Xoncrete: a scheduling tool for partitioned real-time systems, in: ERTS2 2010, Embedded Real Time Software & Systems.

Chan, H.L., Norrish, M., 2019. Proof pearl: Bounding least common multiples with triangles. Journal of Automated Reasoning doi:10.1007/S10817-017-9438-0.

Chandarli, Y., Fauberteau, F., Masson, D., Midonnet, S., Qamhieh, M., 2012. Yartiss: A tool to visualize, test, compare and evaluate real-time scheduling algorithms. Ph.D. thesis. UPE LIGM ESIEE.

Chéramy, M., Hladik, P.E., Déplanche, A.M., 2014. Simso: A simulation tool to evaluate real-time multiprocessor scheduling algorithms, in: 5th international workshop on analysis tools and methodologies for embedded and real-time systems (WATERS), pp. 6–p.

Choquet-Geniet, A., Grolleau, E., 2004. Minimal schedulability interval for real-time systems of periodic tasks with offsets. Theoretical Computer Science 310, 117–134. doi:10.1016/S0304-3975(03)00362-1.

David, A., Illum, J., Larsen, K.G., Skou, A., 2018. Model-based framework for schedulability analysis using uppaal 4.1, in: Model-based design for embedded systems. CRC Press, pp. 117–144.

Diaz, A., Batista, R., Castro, O., 2007. Realtss: a real-time scheduling simulator, in: 2007 4th International Conference on Electrical and Electronics Engineering, IEEE. pp. 165–168.

Garreau, R., Ladeira, M., Grolleau, E., Bauer, H., Ridouard, F., Richard, P., 2023. Link between real-time scheduling and time-triggered networks, in: Proceedings of the 44th IEEE Real-Time Systems Symposium (RTSS 2023), IEEE. pp. 397–410. doi:10.1109/RTSS59052.2023.00041.

Goossens, J., 2003. Scheduling of offset free systems. Real-Time Systems 24, 239–258.

Goossens, J., 2008. (m, k)–firm constraints and dbp scheduling: impact of the initial k-sequence and exact schedulability test, in: 16th International Conference on Real-Time and Network Systems (RTNS 2008).

Goossens, J., Devillers, R., 1997. The non-optimality of the monotonic priority assignments for hard real-time offset free systems. Real-Time Systems doi:10.1023/A:1007980022314.

Goossens, J., Devillers, R., 1999. Feasibility intervals for the deadline driven scheduler with arbitrary deadlines, in: Proceedings of the 6th International Conference on Real-Time Computing Systems and Applications (RTCSA'99), IEEE. pp. 54–61. doi:10.1109/RTCSA.1999.811193.

Goossens, J., Grolleau, E., Cucu-Grosjean, L., 2016. Periodicity of real-time schedules for dependent periodic tasks on identical multiprocessor platforms. Real-Time Systems doi:10.1007/S11241-016-9256-1.

Goossens, J., Masson, D., 2022. Simulation intervals for uniprocessor real-time schedulers with preemption delay, in: Proceedings of the 30th International Conference on Real-Time Networks and Systems (RTNS 2022), ACM. pp. 36–45. doi:10.1145/3534879.3534887.

Grenier, M., Navet, N., 2007. Improvements in the configuration of posix 1003.1 b scheduling, in: 15th International Conference on Real-Time and Network systems-RTNS'2007, pp. 141–150.

Griffin, D., Bate, I., Davis, R.I., 2020. Generating utilization vectors for the systematic evaluation of schedulability tests, in: 2020 IEEE Real-Time Systems Symposium (RTSS), IEEE. pp. 76–88.

Harbour, M.G., Garcia, J.G., Gutiérrez, J.P., Moyano, J.D., 2001. Mast: Modeling and analysis suite for real time applications, in: Proceedings 13th Euromicro Conference on Real-Time Systems, IEEE. pp. 125–134.

Knuth, D.E., 2014. The art of computer programming: Seminumerical algorithms, volume 2. Addison-Wesley Professional.

Ladeira, M., Grolleau, E., Bonneval, F., Hattenberger, G., Ouhammou, Y., Hérouard, Y., 2022a. Scheduling offset-free systems under fifo priority protocol, in: 34th Euromicro Conference on Real-Time Systems (ECRTS 2022). doi:10.4230/LIPIcs.ECRTS.2022.11.

Ladeira, M., Grolleau, E., Bonneval, F., Hattenberger, G., Ouhammou, Y., Hérouard, Y., 2022b. Scheduling Offset-Free Systems Under FIFO Priority Protocol, in: 34th Euromicro Conference on Real-Time Systems (ECRTS 2022), Modena, Italy. URL: https://enac.hal.science/hal-03716151, doi:10.4230/DARTS.8.1.4.

Leung, J.Y.T., Merrill, M.L., 1980. A note on preemptive scheduling of periodic, real-time tasks. Information Processing Letters doi:10.1016/0020-0190(80)90123-4.

Leung, J.Y.T., Whitehead, J., 1982. On the complexity of fixed-priority scheduling of periodic, real-time tasks. Performance Evaluation doi:10.1016/0166-5316(82)90024-4.

Nasri, M., Davis, R.I., Brandenburg, B.B., 2018. Fifo with offsets: High schedulability with low overheads, in: 2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS), IEEE. pp. 271–282.

Prisaznuk, P.J., 2008. Arinc 653 role in integrated modular avionics (ima), in: 2008 IEEE/AIAA 27th digital avionics systems conference, IEEE. pp. 1–E.

Singhoff, F., Legrand, J., Nana, L., Marcé, L., 2004. Cheddar: a flexible real time scheduling framework, in: Proceedings of the 2004 annual ACM SIGAda international conference on Ada: The engineering of correct and reliable software for real-time & distributed systems using Ada and related technologies, pp. 1–8.

Steiner, W., 2010. An evaluation of SMT-based schedule synthesis for time-triggered multi-hop networks, in: 31st IEEE Real-Time Systems Symposium 2010, IEEE. doi:10.1109/RTSS.2010.25.

Urunuela, R., Déplanche, A.M., Trinquet, Y., 2010. Storm a simulation tool for real-time multiprocessor scheduling evaluation, in: 2010 IEEE 15th Conference on Emerging Technologies & Factory Automation (ETFA 2010), IEEE. pp. 1–8.